Here's a **complete TypeScript example** that takes a **day, month, year** and a **timezone** string, and converts it into a **full ISO 8601 date-time** string for that timezone.

We'll use the `Intl.DateTimeFormat` API with `timeZone` and then adjust to ISO format.

```typescript
// TypeScript: Convert day, month, year to full ISO date-time in a given timezone

/**
 * Converts a given day, month, year to an ISO 8601 date-time string
 * for the specified IANA timezone (e.g., "Europe/Rome").
 *
 * @param day - Day of the month (1-31)
 * @param month - Month (1-12)
 * @param year - Full year (e.g., 2026)
 * @param timeZone - IANA timezone string (e.g., "Europe/Rome")
 * @returns ISO 8601 date-time string with timezone offset applied
 */
function toISODateTime(
  day: number,
  month: number,
  year: number,
  timeZone: string
): string {
  // Validate inputs
  if (
    !Number.isInteger(day) || day < 1 || day > 31 ||
    !Number.isInteger(month) || month < 1 || month > 12 ||
```

```
  !Number.isInteger(year) || year < 1
) {
  throw new Error("Invalid date components provided.");
}

try {
  // Create a UTC date from the given components
  const utcDate = new Date(Date.UTC(year, month - 1, day, 0, 0, 0));

  // Use Intl.DateTimeFormat to get the local time in the given timezone
  const formatter = new Intl.DateTimeFormat("en-CA", {
    timeZone,
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
    hour: "2-digit",
    minute: "2-digit",
    second: "2-digit",
    hour12: false
  });

  // Extract parts
  const parts = formatter.formatToParts(utcDate);
  const get = (type: string) => parts.find(p => p.type === type)?.value || "00";

  const localDateTime = `${get("year")}-${get("month")}-${get("day")}T${get("hour")}:
```

```
${get("minute")}:${get("second")}`;

    // Get the timezone offset in ±HH:MM format
    const offsetMinutes = -(
      new Date(localDateTime + "Z").getTime() - utcDate.getTime()
    ) / 60000;
    const sign = offsetMinutes >= 0 ? "+" : "-";
    const pad = (n: number) => String(Math.floor(Math.abs(n))).padStart(2, "0");
    const offset = `${sign}${pad(offsetMinutes / 60)}:${pad(offsetMinutes % 60)}`;

    return `${localDateTime}${offset}`;
  } catch (err) {
    throw new Error(`Failed to convert date: ${(err as Error).message}`);
  }
}

// Example usage:
const isoString = toISODateTime(24, 4, 2026, "Europe/Rome");
console.log(isoString);
// Example output: "2026-04-24T00:00:00+02:00"
```

How it works:

1. **Input Validation** – Ensures day, month, and year are valid integers.
2. **UTC Base Date** – Creates a `Date` object in UTC to avoid local machine bias.

3. `Intl.DateTimeFormat` – Formats the date in the given timezone.
4. **Offset Calculation** – Computes the correct timezone offset in `±HH:MM` format.
5. **ISO String Assembly** – Combines date, time, and offset into a valid ISO 8601 string.

If you want, I can also give you a **simpler version using** `Temporal` **API** (which is cleaner and avoids manual offset math) — but it requires Node.js 20+ or a polyfill.

Do you want me to provide that modern `Temporal` version?